

RepairAgent: An Autonomous, LLM-Based Agent for Program Repair

ISLEM BOUZENIA, University of Stuttgart, Germany

PREMKUMAR DEVANBU, UC Davis, USA

MICHAEL PRADEL, University of Stuttgart, Germany

Automated program repair has emerged as a powerful technique to mitigate the impact of software bugs on system reliability and user experience. This paper introduces RepairAgent, the first work to address the program repair challenge through an autonomous agent based on a large language model (LLM). Unlike existing deep learning-based approaches, which prompt a model with a fixed prompt or in a fixed feedback loop, our work treats the LLM as an agent capable of autonomously planning and executing actions to fix bugs by invoking suitable tools. Repair Agent freely interleaves gathering information about the bug, gathering repair ingredients, and validating fixes, while deciding which tools to invoke based on the gathered information and feedback from previous fix attempts. Key contributions that enable RepairAgent include a set of tools that are useful for program repair, a dynamically updated prompt format that allows the LLM to interact with these tools, and a finite state machine that guides the agent in invoking the tools. Our evaluation on the popular Defects4J dataset demonstrates RepairAgent’s effectiveness in autonomously repairing 164 bugs, including 39 bugs not fixed by prior techniques. Interacting with the LLM imposes an average cost of 270k tokens per bug, which, under the current pricing of OpenAI’s GPT-3.5 model, translates to 14 cents per bug. To the best of our knowledge, this work is the first to present an autonomous, LLM-based agent for program repair, paving the way for future agent-based techniques in software engineering.

ACM Reference Format:

Islem Bouzenia, Premkumar Devanbu, and Michael Pradel. 2026. RepairAgent: An Autonomous, LLM-Based Agent for Program Repair. 1, 1 (July 2026), 14 pages. <https://doi.org/10.1145/nnnnnnn.nnnnnnn>

1 Introduction

Software bugs can lead to system failures, security vulnerabilities, and compromised user experience. Manually fixing bugs demands significant time and user effort, but **Automated Program Repair (APR)** can significantly reduce the effort via automatic bug resolution. Various approaches have been explored in this regard [Le Goues et al. 2019], incl. techniques based on manually designed [Le Goues et al. 2012], [Liu et al. 2019] and (semi-)automatically extracted [Kim et al. 2013], [Bader et al. 2019] fix patterns, based on symbolic constraints [Nguyen et al. 2013], [Xuan et al. 2016], and various machine learning based approaches [Long and Rinard 2016], [Gupta et al. 2017], [Tufano et al. 2019].

The current state-of-the-art in APR is predominated by LLMs. In the first generation of LLM based APR, the model expects a prompt with the buggy code and produces the fixed version [Xia et al. 2023], [Jiang et al. 2023], but further development introduces an iterative approach: the LLM is queried repeatedly based on feedback obtained from previous fix attempts [Xia and Zhang 2023], [Kang et al. 2023a], [Ye and Monperrus 2024].

But this approach has a few limitations since the hard-coded feedback loops in the LLMs do not allow them to gather information about existing bugs or existing code that may provide them with debugging ingredients, unlike manual

Authors’ Contact Information: Islem Bouzenia, fi_bouzenia@esi.dz, University of Stuttgart, Germany; Premkumar Devanbu, ptdevanbu@ucdavis.edu, UC Davis, USA; Michael Pradel, michael@binaervarianz.de, University of Stuttgart, Germany.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

© 2026 Copyright held by the owner/author(s). Publication rights licensed to ACM.

Manuscript submitted to ACM

human debugging which typically incorporates a temporal interleaving of gathering information to gain understanding of the bug, searching code that might help in debugging and then attempting various versions of fixes [Ko et al. 2006], [Böhme et al. 2017].

This paper represents **RepairAgent**, the first autonomous, LLM based agent for automated program repair. The LLM is treated as an autonomous agent capable of planning and executing actions to fix a bug. The LLM is equipped with a set of repair-specific tools. How and when to use these tools aren't hard coded, rather the LLM gets to autonomously decide.

The approach is enabled via three key components: a general purpose LLM e.g. GPT-3.5, a set of tools for the LLM to use (in this approach, 14 tools in total have been used), and a middleware to establish communication between the tools and the LLM.

The novelty of the approach was tested using all 835 bugs of the Defects4J [Just et al. 2014] dataset. 164 bugs were successfully fixed, including 39 bugs that were not fixed by prior state-of-the-art work [Xia and Zhang 2023] [Ye and Monperrus 2024].

The interaction with the LLM imposes a cost of $\sim 270K$ tokens per bug on average, which translates to about **14 cents** per bug.

In summary, this paper contributes the following:

- An autonomous, LLM-based agent for program repair.
- A dynamically updated prompt format that guides the LLM through the bug repair process.
- A set of tools that enable a LLM to perform steps a human developer would take when fixing a bug.
- A middleware that guides via a finite state machine how the LLM interacts with the tools.
- Empirical evidence that RepairAgent establishes a new state of the art in program repair.
- We release our implementation as open-source: <https://github.com/sola-st/RepairAgent>.

To the best of our knowledge, this is the first work to present an autonomous, LLM-based agent for program repair, paving the way for future agent-based techniques in software engineering.

2 Background (LLM-Based Autonomous Agents)

Because of extensive training, LLMs have demonstrated remarkable abilities in achieving human-level performance in a variety of tasks [Bubeck et al. 2023]. A promising way to use these LLMs are LLM-based agents, which mean LLM-based techniques with two properties: (1) The LLM can autonomously plan for and execute a set of actions to achieve a particular goal, unlike hard-coded algorithmic processes, and (2) The actions by the LLM can invoke external tools, enabling it to interact with its environment [Schick et al. 2023] [Patil et al. 2023]. In the context of Software Engineering, the tools could be tools usually used by developers, e.g., as part of an integrated development environment (IDE). Basically, we query the LLM with the current state of the world, the goal and a set of actions to perform. The feedback is integrated into the next prompt.

3 Method

3.1 Overview

Fig. 1 provides an overview of RepairAgent.

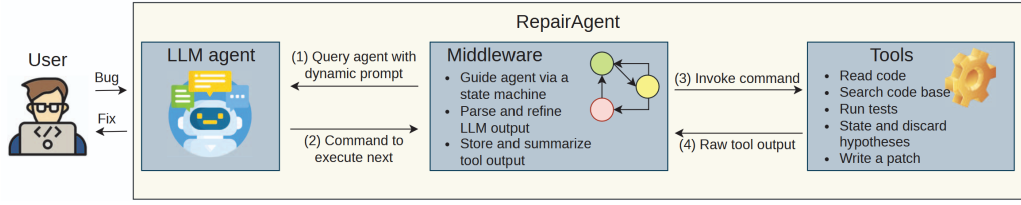


Fig. 1. Overview of RepairAgent.

3.2 Terminology

RepairAgent proceeds in multiple cycles:

Definition 1 (Cycle). A *cycle* represents one round of interaction with the LLM agent, which consists of the following steps:

- (1) Query the agent
- (2) Post-process the response
- (3) Execute the command suggested by the agent
- (4) Update the dynamic prompt based on the command's output

In each cycle, the approach queries the LLM once. The input to the model is updated based on commands invoked by the LLM, and their results, in previous cycles. We call the model input a dynamic prompt:

Definition 2 (Dynamic prompt). The *dynamic prompt* is a sequence of text sections $P = [s_0, s_1, \dots, s_n]$, where each section s_i is one of the following (where $s_i(c)$ refers to a section during a cycle c):

- A *static section*, which remains the same across all cycles, i.e., $s_i(c) = s_i(c')$ for all c, c' .
- A *dynamic section*, which may differ across cycles, i.e., there may exist c, c' with $s_i(c) \neq s_i(c')$.

3.3 Dynamic Prompting of the RepairAgent

As listed in Table 1, the prompt contains multiple static and dynamic sections.

Table 1. Sections of the dynamically updated prompt within RepairAgent

Prompt Section	Nature
Role	Static
Goals	Static
Guidelines	Static
State description	Dynamic
Available tools	Dynamic
Gathered information	Dynamic
Specification of output format	Static
Last executed command and result	Dynamic

(1) **Role**: provides the agent's area of expertise and outlines the agent's primary objective: finding and fixing bugs

(2) **Goals**: There are 5 goals for the agent to pursue:

- a) *Locate the bug* using testing and fault localization techniques

- b) *Gather information about the bug* by analyzing the relevant portion(s) of the codebase
- c) *Suggest simple fixes to the bug*
- d) *Suggest complex fixes*
- e) *Iterate over the previous goals*

(3) Guidelines: The LLM is provided with a set of guidelines. It is told that there are various kinds of bugs ranging from one-line-fix bugs to multi-line bugs. A list of recurring fix patterns is also provided that help solve many common bugs [Karampatsis and Sutton 2020] which is based on single-statement bugs in **Java**. The model is also instructed to add comments along with edited code which helps the model explain its reasoning and improve itself [Wei et al. 2022]. The model is also instructed to define a next step that can be converted into a call to a tool.

(4) State description: To guide the LLM toward using the tools effectively, a state machine is defined that constrains which tool(s) can be used at any given time. This is to stop the LLM to get lost in aimless exploration, which has happened very frequently as per the results of earlier tests. Fig. 2 shows the finite state machine. The set of tools are described in Section 3.4. The agent is free to move between any state via tools.

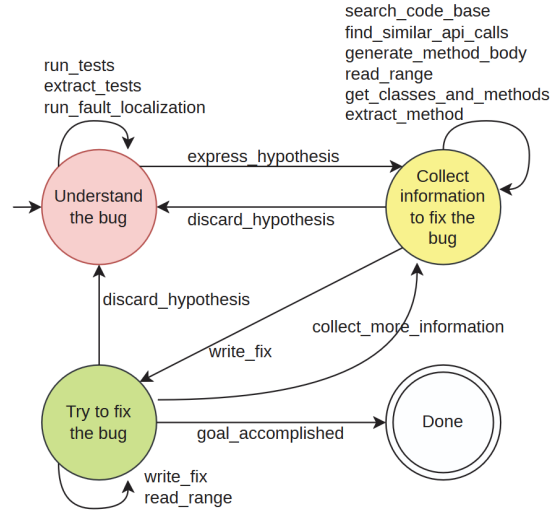


Fig. 2. State machine to guide selection of tools.

- *Understand the bugs:* The agent can collect information and bug location from testing results. It can then formulate a hypothesis based on its knowledge.
- *Collect information to fix the bug:* The agent can search for specific repair methods or read relevant code. After gathering enough information, it moves on to the next state.
- *Try to fix the bug:* The agent can try to fix the bug based on the hypothesis and information gathered. If necessary, the agent can go back and establish a new hypothesis or gather additional information.

(5) Available tools: This section describes the currently available tools at any state.

(6) Gathered information: This serves as a memory for the agent, enabling it to recall information from previous cycles. This section is divided into various subsections that contain outputs from different tools.

(7) **Specification of output format:** The agent gives one response per cycle in the format specified by Fig. 3. The "thoughts" section is the agent's reasoning when deciding what to do for the next command.

```
interface Response {
  thoughts: string;
  command: {
    name: string;
    args: Record<string, any>;
  };
}
```

Fig. 3. Output format specification.

```
{
  "thoughts": "With the hypothesis in mind,
it's time to gather more information to
formulate potential fixes for the bug. I
should focus on understanding the context
around the condition
'if (x < 0 && prev == '-')'
to come up with effective solutions.",
  "command": {
    "name": "search_code_base",
    "args": {
      "key_words":
        [ "addNumber",
          "CodeConsumer",
          "if (x < 0 && prev == '-')" ]
    }
  }
}
```

Fig. 4. Example agent response layout.

(8) **Last executed command and result:** This section helps the agent remind the last steps taken in case anything went wrong during execution. It also helps the agent remember how many cycles have been executed and how many are left.

3.4 Tools for the Agent to Use

A key novelty of the approach is letting the LLM agent autonomously decide which tools to invoke to fix a bug. The tools provided to the agent (Table 2) are inspired by the tools developers typically use in their IDEs.

- a) *Reading and Extracting Code:* Instead of hard-coding the context provided to the LLM [Xia and Zhang 2023], [Kang et al. 2023a], [Ye and Monperrus 2024], the agent gets to decide which parts of the code to read. It uses tools like `read_range` to extract specific lines for a focused view, `get_classes_and_methods` for a code structure overview, `extract_method` to retrieve method implementations, and `extract_tests` to get the code of failing test cases to understand expected outputs.
- b) *Search and Generate Code:* Because human developers commonly search for code [De Grazia and Pradel 2023], the agent is equipped with tools to search for specific code snippets to gather repair ingredients. The `search_code_base` tool locates keyword instances across the entire codebase using approximate matching (splitting keywords into subtokens). The `find_similar_api_calls` tool extracts method usages to prevent the LLM from hallucinating non-existent method calls [Eghbali and Pradel 2024]. Additionally, a `generate_method_body` tool can generate new code by invoking another LLM, inspired by the success of code completion tools like Copilot [Chen et al. 2021a].
- c) *Testing and Patching:* `run_tests` executes the project's test suite and cleans the output to avoid confusing the model with irrelevant stack-trace information. `run_fault_localization` retrieves fault-localization data, either assuming perfect localization or using tools like GZoltar [Campos et al. 2012]. `write_fix` applies a JSON patch and runs the test suite; if tests fail, it reverts the changes to give the agent a clean slate. The tool also samples multiple variants (30 by default) of a suggested fix to account for almost-correct attempts.

Table 2. Repair-related tools invoked by RepairAgent.

Tool	Description
Read and extract code:	
<code>read_range</code>	Read a range of lines in a file.
<code>get_classes_and_methods</code>	Get the names of all classes and methods in a file.
<code>extract_method</code>	Given a method name, extract method implementations from a file.
<code>extract_tests</code>	Given the failure report from JUnit or ANT, extract the code of failing test cases.
Search and generate code:	
<code>search_code_base</code>	Scans all Java files within a project for a list of keywords.
<code>find_similar_api_calls</code>	Given a code snippet that calls an API, search for similar API calls in the project.
<code>generate_method_body</code>	Ask an LLM (GPT-3.5 by default) to generate the body of a method based on code proceeding the method.
Testing and patching:	
<code>run_tests</code>	Run the test suite of a project.
<code>run_fault_localization</code>	Retrieve pre-existing localization information or run a fault localization tool.
<code>write_fix</code>	Apply a patch to the code base and execute the test suite of the project. Changes are reverted automatically if tests fail. Moves the agent into the “Try to fix the bug” state.
Control:	
<code>express_hypothesis</code>	Express a hypothesis about the bug. Moves the agent into the “Collect information to fix the bug” state.
<code>collect_more_information</code>	Move the agent back to the “Collect information to fix the bug” state.
<code>discard_hypothesis</code>	Discard the current hypothesis about the bug and move back to the “Understand the bug” state.
<code>goal_accomplished</code>	Declare that the goal has been accomplished and exiting the repair process.

- d) *Control*: These tools don’t directly correspond to human developer tools, rather they allow the agent to navigate the finite state machine. `express_hypothesis` and `discard_hypothesis` enforce a structured approach to hypothesis formulation, aligning with scientific debugging [Zeller 2009], [Kang et al. 2023a]. `collect_more_information` reverts the agent to gathering mode if fixes fail, and `goal_accomplished` terminates the process once a passing fix is found.

3.5 middleware

The middleware component orchestrates the communication between the LLM agent and the tools, seamlessly executing the steps of each cycle. It handles three primary responsibilities:

- (1) **Parsing and Refining LLM Output**: At the start of a cycle, the middleware queries the LLM. Because LLMs can produce responses with syntactic errors or hallucinations (e.g., using “path” instead of “file_path”), the middleware heuristically rectifies these deviations in three steps:
 - Maps the predicted tool name to an actual tool using substring matching or a Levenshtein distance threshold (0.1 by default).
 - Maps the provided argument names using the same matching logic.
 - Heuristically corrects invalid argument values, such as replacing a broken file path with a valid one.

If these heuristics fail, or if the agent suggests the exact same command as a previous cycle, the middleware informs the LLM about the error via the “Last executed command and result” prompt section and initiates a new cycle.

- (2) **Calling the Tool:** Once a valid command is parsed, the middleware executes the corresponding tool in an isolated environment to prevent any interference with the host system or RepairAgent itself.
- (3) **Updating the Prompt:** Using the tool’s output, the middleware updates all dynamic sections of the prompt for the next cycle. This includes refreshing the state description and available tools, appending the new output to the gathered information, and updating the last executed command section.

4 Implementation

We use Python 3.10 as our primary programming language. Docker is used to containerize and isolate command executions for enhanced reliability and reproducibility. RepairAgent is built on top of the AutoGPT framework and GPT-3.5-0125 from OpenAI. To parse and interact with Java code, we use ANTLR.

5 Findings/Results

To evaluate the approach, RepairAgent was tested to answer four main research questions regarding its effectiveness, costs, component importance, and tool usage.

5.1 Experimental Setup

- **Datasets:** Evaluated on all 835 bugs from the Defects4J [Just et al. 2014] dataset (v1.2 and v2.0). To assess generalization and mitigate data leakage risks, a random sample of 100 bugs from the newer GitBug-Java [Silva et al. 2024] dataset was also used.
- **Baselines:** Compared against three state-of-the-art iterative repair techniques: ChatRepair [Xia and Zhang 2023], ITER [Ye and Monperrus 2024], and SelfAPR [Ye et al. 2022b].
- **Metrics:** Evaluated based on *plausible* patches (passes all tests) and *correct* patches (syntactically matches or is semantically consistent with the developer’s fix).

5.2 Effectiveness (RQ1)

Evaluations performed across the Defects4J benchmark indicate that RepairAgent effectively resolved **164 functional bugs** (186 plausible), establishing capabilities beyond baseline tools like ChatRepair and ITER.

As shown in Table 3, RepairAgent particularly excels in Defects4Jv2, fixing 90 bugs compared to ChatRepair’s 48. Furthermore, it successfully fixed **39 unique bugs** that were not fixed by any of the prior state-of-the-art baselines (see Fig. 4).

Table 3. Results on Defects4J.

Project	Bugs	Plausible	Correct	ChatRepair	ITER
Chart	26	14	11	15	10
Closure	174	27	27	37	18
Math	106	29	29	32	0
Defects4Jv1.2	395	96	74	114	57
Defects4Jv2	440	90	90	48	—
Total	835	186	164	162	57

This performance is especially visible in multi-line fixes due to the contextual integration of search features. Table 4 shows RepairAgent outperforming baselines on complex, multi-line bugs. Further cross-validation against the GitBug-Java set shows functional adaptation across varied setups, successfully finding 13 correct fixes [Silva et al. 2024].

Table 4. Distribution of fixes by location type

Bug type	RepairAgent	ChatRepair	ITER	SelfAPR
Single-line	115	133	36	83
Multi-line*	46	29	14	24
Multi-file	3	0	4	3

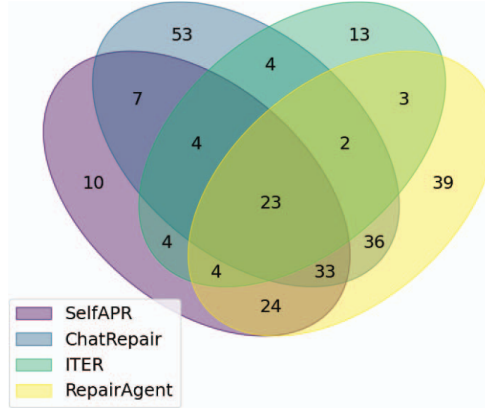


Fig. 4. Intersection of the fixes set with related work.

5.3 Costs of the Approach (RQ2)

Financial and temporal evaluations tracking token usage show an average financial overhead of approximately **14 cents per bug** resolution attempt (around 270k tokens) under standard GPT-3.5 pricing.

The median time taken to address a bug is 920 seconds (Fig. 5a). This is highly efficient compared to prior work; for instance, ITER reports a median bug fixing time of 4.57 hours per bug [Ye and Monperrus 2024].

5.4 Ablation Study (RQ3)

To understand the impact of different configurations, an ablation study was performed on a random set of 100 Defects4J bugs (Table 5).

- **No search tools:** Fixes drop by half, and costs double due to the agent reading long sequences of code instead of searching.
- **No state machine:** The agent loses structure, fixing fewer bugs and increasing costs.
- **Single-cycle memory:** Removing long-term memory causes a massive drop in effectiveness (only 6 correct fixes) as the agent repeats commands and forgets file names.
- **Realistic localization:** Using GZoltar [Campos et al. 2012] instead of perfect fault localization drops fixing capability by 25% and increases costs, as the agent spends extra time localizing the bug.

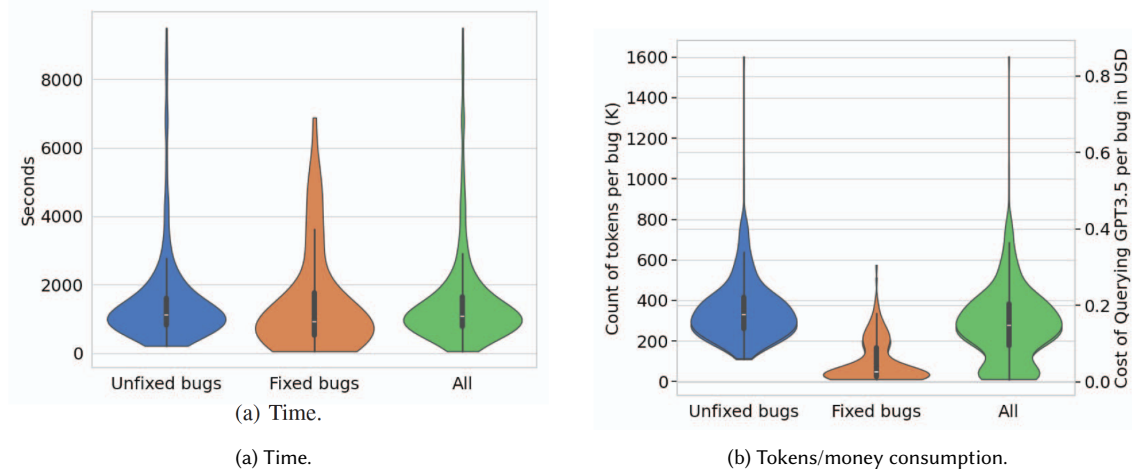


Fig. 5. Distribution of cost metrics per bug.

Table 5. Different configurations of RepairAgent.

Configuration	Plausible	Correct	Cost(\$)
No search tools	14	11	0.28
No state machine	18	14	0.31
Single-cycle memory	9	6	0.08
Realistic localization	16	16	0.29
RepairAgent (default)	23	21	0.16

5.5 Usage of Tools (RQ4)

On average, RepairAgent makes 35 tool invocations per bug. Fig. 6 shows that *write_fix* is the most frequently called tool. Interestingly, *run_tests* is used infrequently because the *write_fix* tool automatically invokes the test suite upon applying a patch.

6 Discussion

6.1 Qualitative Insights

Qualitative tracing indicates that access to structural tools yields better insights into functional anomalies. The agent’s ability to actively retrieve information—such as the code of failing tests, snippets retrieved via *find_similar_api_calls*, and feedback from applied fixes—allowed it to fix a new set of complex bugs without higher costs.

However, complex modifications sometimes introduce unintended changes to straightforward single-line defects. While RepairAgent clearly outperforms prior work on multi-line bugs, it occasionally fails to fix simpler, single-line bugs because the agent overcomplicates the solution. A potential remedy for future work is to initially limit the complexity of candidate fixes to nudge the agent toward simple solutions first.

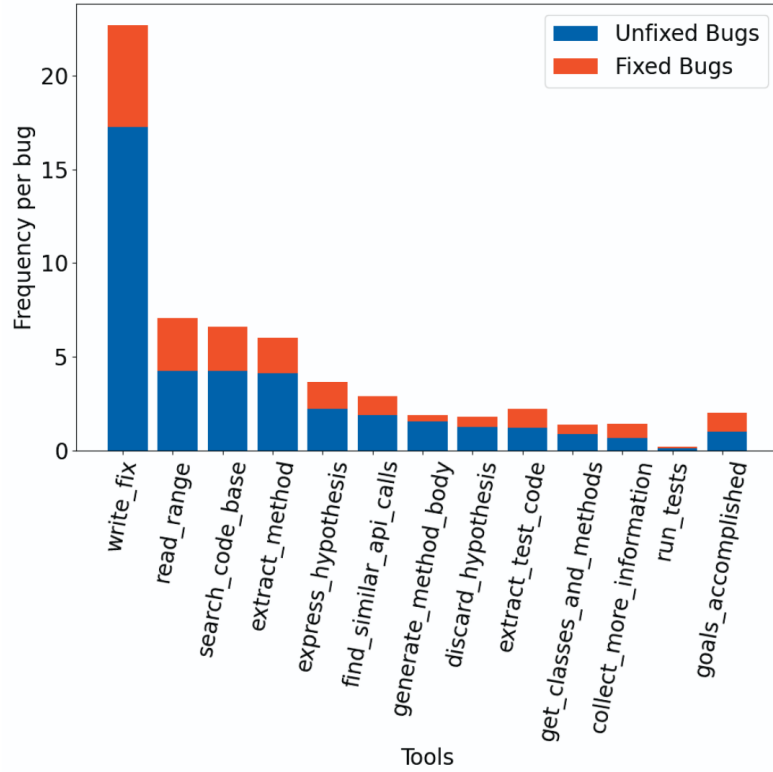


Fig. 6. Frequency of tool invocations (average per bug).

6.2 Threats to Validity and Limitations

- **Data Leakage:** Foundational models like GPT-3.5 may have seen parts of the Defects4J Java projects during pre-training. This risk is shared with competitors like ChatRepair [Xia and Zhang 2023], but is mitigated by our evaluation on the newer GitBug-Java dataset [Silva et al. 2024].
- **Missing Test Cases:** Defects4J guarantees at least one failing test case per bug, which may not reflect real-world scenarios where error-revealing tests are unavailable.
- **Fault Localization:** The default setup relies on perfect fault localization. As shown in the ablation study, relying on secondary spectrum coverage frameworks like GZoltar [Campos et al. 2012] introduces imprecision that can lead to suboptimal repair suggestions.
- **Non-deterministic Outputs:** The inherently non-deterministic nature of LLMs means outcomes may vary between consecutive runs.

7 Related Work

- **Non-learning-based program repair:** Automated program repair (APR) [Le Goues et al. 2019] has a rich history. Traditional approaches treat it as a search problem using manually designed mutation rules and fix patterns [Le et al. 2016; Le Goues et al. 2012; Liu et al. 2019]. Others derive rules from human-written patches

[Bader et al. 2019; Bavishi et al. 2019; Kim et al. 2013], utilize symbolic constraints [Ke et al. 2015; Mechtaev et al. 2016; Nguyen et al. 2013; Xuan et al. 2016], or integrate repair into static analysis [Jain et al. 2023; Liu et al. 2023; van Tonder and Le Goues 2018] and code transplantation [Yang et al. 2022]. APR has even seen industrial deployment [Bader et al. 2019; Marginean et al. 2019]. Beyond functional bugs, techniques exist for syntax errors [Gupta et al. 2019; Sakkas et al. 2022; Wang et al. 2018], performance bottlenecks [Yu and Pradel 2017], vulnerabilities [Harer et al. 2018], type errors [Chow et al. 2024], deep learning issues [Zhang et al. 2021], and build errors [Tarlow et al. 2020].

- **Learning-based program repair:** Early ML approaches focused on ranking candidate fixes [Long and Rinard 2016]. Recent work shifted to generation, using Neural Machine Translation (NMT) to map buggy code to fixed code [Chen et al. 2021b; Gupta et al. 2017; Lutellier et al. 2020; Tufano et al. 2019], predicting tree transformations [Li et al. 2020; Zhu et al. 2021], and employing repair-specific neural architectures and training regimes [Vasic et al. 2019; Ye et al. 2022b,a]. For a comprehensive overview, see [Zhang et al. 2023]. Unlike these, RepairAgent uses a general-purpose LLM rather than a task-specific model.
- **LLMs for APR and SE:** LLMs are increasingly applied to APR via one-time prompt interactions [Jiang et al. 2023; Joshi et al. 2023; Xia et al. 2023]. The current state-of-the-art relies on iterative feedback loops [Hidvégi et al. 2024; Kang et al. 2023a; Xia and Zhang 2023; Ye and Monperrus 2024]. Beyond repair, LLMs are used for code completion [Chen et al. 2021a; Shrivastava et al. 2023], fuzzing [Xia et al. 2024], unit testing [Alshahwan et al. 2024; Feng and Chen 2024; Kang et al. 2023b; Lemieux et al. 2023; Ryan et al. 2024; Schäfer et al. 2024], and multi-step editing [Bairi et al. 2023]. However, **none of these approaches are agent-based**. RepairAgent fundamentally differs by letting the model autonomously plan and execute actions.
- **LLM-based agents:** The concept of LLMs autonomously planning and performing complex tasks is relatively new [Wang et al. 2023]. RepairAgent is inspired by prior work on augmenting LLMs with API tools [Mialon et al. 2023; Patil et al. 2023; Schick et al. 2023] and code execution capabilities [Gao et al. 2022]. To the best of our knowledge, **RepairAgent is the first LLM-based agent applied to program repair**. Following the initial public release of this work, other SE agents have emerged, such as Copra for theorem proving [Thakur et al. 2024], AutoCodeRover [Zhang et al. 2024], and SWE-agent [Yang et al. 2024], highlighting the massive potential of this paradigm.

8 Conclusion

This paper presents a pioneering technique for bug repair based on an autonomous agent powered by Large Language Models (LLMs). By equipping the LLM with IDE-like tools and a dynamic prompt structure, we treat the model not as a simple text generator, but as an active participant in the debugging process. Through extensive experimentation on the Defects4J benchmark, we validate the effectiveness and financial viability of our approach. Further exploration and refinement of autonomous agent-based techniques—specifically by equipping them with the right tools—will help generalize automated repair to even more difficult and diverse types of software bugs.

Data Availability

Our code and data are available at: <https://github.com/sola-st/RepairAgent>

Alternatively: <https://doi.org/10.5281/zenodo.14470521>

Acknowledgments

This work was supported by the European Research Council (ERC, grant agreements 851895 and 101155832), and by the German Research Foundation within the ConcSys, DeMoCo, and QPTest projects.

References

- Nadia Ext Alshahwan, Janki Ext Chheda, Alina Ext Ext Finegenova, Birol Ext Gokkaya, Mark Harman, Inna Ext Harper, Alexandru Marginean, Sukriti Ext Sengupta, and Eddy Ext Wang. 2024. Automated unit test improvement using large language models at meta. *FSE abs/2402.09171* (2024). doi:10.48550/arXiv.2402.09171
- Johannes Bader, Andrew Scott, Michael Pradel, and Satish Chandra. 2019. Getafix: Learning to fix bugs automatically. *Proc. ACM Program. Lang.* 3, OOPSLA (2019), 159:1–159:27. doi:10.1145/3360585
- Ramakrishna Ext Bairi, Abhishek Ext Sonwane, Aditya Ext Kanade, Chandan Ext Ext V. D., Arun Ext Iyer, Sriram Ext Parthasarathy, Sriram Ext Rajamani, B. Ext Ashok, and Shashank Ext Shet. 2023. Codeplan: Repository-level coding using llms and planning. *arXiv preprint* (2023).
- Rohan Bavishi, Hiroaki Yoshida, and Mukul R. Prasad. 2019. Phoenix: automated data-driven synthesis of repairs for static analysis violations. In *Proceedings of the 27th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE)*. 613–624. doi:10.1145/3338906.3338952
- Marcel Böhme, Ezekiel O. Soremekun, Sudipta Chattopadhyay, Ezekiel Ugherughe, and Andreas Zeller. 2017. Where is the bug and how is it fixed? an experiment with practitioners. In *Proceedings of the 2017 11th Joint Meeting on Foundations of Software Engineering (ESEC/FSE)*. 117–128.
- Sébastien Bubeck, Varun Ext Chandrasekaran, Ronen Ext Eldan, Johannes Gehrke, Eric Horvitz, Ece Ext Kamar, Peter Lee, Yin Tat Lee, Yuanzhi Ext Li, Scott M. Lundberg, Harsha Ext Nori, Hamid Ext Palangi, Marco Tulio Ribeiro, and Yi Zhang. 2023. Sparks of artificial general intelligence: Early experiments with GPT-4. *CoRR abs/2303.12712* (2023). doi:10.48550/arXiv.2303.12712
- José Ext Campos, André Ext Ribeiro, Alexandre Ext Perez, and Rui Ext Abreu. 2012. Gzoltar: an eclipse plug-in for testing and debugging. In *Proceedings of the 27th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. 378–381.
- Mark Chen et al. 2021a. Evaluating large language models trained on code. *CoRR abs/2107.03374* (2021). <https://arxiv.org/abs/2107.03374>
- Zimin Chen, Steve Kommrusch, Michele Tufano, Louis-Noël Pouchet, Denys Poshyvanyk, and Martin Monperrus. 2021b. SequenceR: Sequence-to-sequence learning for end-to-end program repair. *IEEE Trans. Software Eng.* 47, 9 (2021), 1943–1959. doi:10.1109/TSE.2019.2940179
- Yin Wei Ext Chow, Luca De Grazia, and Michael Pradel. 2024. Pyty: Repairing static type errors in python. In *International Conference on Software Engineering (ICSE)*.
- Luca De Grazia and Michael Pradel. 2023. Code search: A survey of techniques for finding code. *ACM Comput. Surv.* 55, 11 (2023), 220:1–220:31. doi:10.1145/3565971
- Ali Ext Eghbali and Michael Pradel. 2024. De-hallucinator: Iterative grounding for llm-based code completion. *CoRR abs/2401.01701* (2024). doi:10.48550/arXiv.2401.01701
- Sidong Ext Feng and Chun Ext Ext Chen. 2024. Prompting is all your need: Automated android bug replay with large language models. In *Proceedings of the International Conference on Software Engineering (ICSE)*.
- Luyu Ext Gao, Aman Ext Ext Madaan, Shuyan Ext Ext Ext Zhou, Uri Ext Alon, Pengfei Ext Liu, Yiming Ext Yang, Jamie Ext Callan, and Graham Neubig. 2022. PAL: program-aided language models. *CoRR abs/2211.10435* (2022). doi:10.48550/arXiv.2211.10435
- Rahul Gupta, Aditya Kanade, and Shirish K. Shevade. 2019. Deep reinforcement learning for syntactic error repair in student programs. In *Proceedings of the Thirty-Third AAAI Conference on Artificial Intelligence*. 930–937. doi:10.1609/aaai.v33i01.3301930
- Rahul Gupta, Soham Pal, Aditya Kanade, and Shirish K. Shevade. 2017. Deepfix: Fixing common C language errors by deep learning. In *Proceedings of the AAAI Conference on Artificial Intelligence*. 1345–1351. <http://aaai.org/ocs/index.php/AAAI/AAAI17/paper/view/14603>
- Jacob Ext Harer, Onur Ext Ozdemir, Tom Ext Lazovich, Christopher P. Ext Reale, Rebecca L. Ext Russell, Louis Y. Ext Kim, and Steve P. Ext Chin. 2018. Learning to repair software vulnerabilities with generative adversarial networks. In *Advances in Neural Information Processing Systems* 31. 7944–7954.
- Dánid Ext Hidvégi, Khashayar Ext Ettemadi, Sofia Ext Bobadilla, and Martin Monperrus. 2024. Cigar: Cost-efficient program repair with llms. *arXiv preprint arXiv:2402.06598* (2024).
- Niranjan Ext Jain, Saikat Ext Gandhi, Abhishek Ext Sonwane, Aditya Kanade, Nagarajan Ext Natarajan, Sriram Ext Parthasarathy, Sriram Ext Rajamani, and Rahul Ext Sharma. 2023. Staticfixer: From static analysis to static repair. *arXiv preprint* (2023).
- Nan Ext Jiang, Kui Liu, Thibaud Lutellier, and Lin Tan. 2023. Impact of code language models on automated program repair. In *Proceedings of the International Conference on Software Engineering (ICSE)*. 1430–1442. doi:10.1109/ICSE48619.2023.00125
- Harshit Ext Joshi, José Pablo Cambronero Ext Ext Sánchez, Sumit Gulwani, Vu Ext Ext Le, Gust Ext Verbruggen, and Ivan Ext Radicek. 2023. Repair is nearly generation: Multilingual program repair with llms. In *Thirty-Seventh AAAI Conference on Artificial Intelligence*. AAAI Press, 5131–5140. doi:10.1609/aaai.v37i4.25642
- René Just, Darioush Jalali, and Michael D. Ernst. 2014. Defects4j: a database of existing faults to enable controlled testing studies for java programs. In *Proceedings of the 2014 International Symposium on Software Testing and Analysis (ISSTA)*. 437–440.
- Sungmin Kang, Bei Chen, Shin Yoo, and Jian-Guang Lou. 2023a. Explainable automated debugging via large language model-driven scientific debugging. *CoRR abs/2304.02195* (2023). doi:10.48550/arXiv.2304.02195

Manuscript submitted to ACM

- Sungmin Kang, Juseon Ext Yoon, and Shin Yoo. 2023b. Large language models are few-shot testers: Exploring llm-based general bug reproduction. In *45th IEEE/ACM International Conference on Software Engineering (ICSE)*. 2312–2323. doi:10.1109/ICSE48619.2023.00194
- Rahul-Marie Karampatsis and Charles Sutton. 2020. How often do single-statement bugs occur? (Jun. 2020). doi:10.1145/3379597.3387491
- Yalin Ext Ke, Kathryn T. Stolee, Claire Le Goues, and Yuriy Brun. 2015. Repairing programs with semantic code search (t). In *Proceedings of the 30th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. IEEE, 295–306.
- Dongsun Kim, Jaechang Nam, Jaewoo Song, and Sunghun Kim. 2013. Automatic patch generation learned from human-written patches. In *Proceedings of the International Conference on Software Engineering (ICSE)*. 802–811.
- Andrew J. Ko, Brad A. Myers, Michael J. Coblenz, and Htet Htet Aung. 2006. An exploratory study of how developers seek, relate, and collect relevant information during software maintenance tasks. *IEEE Transactions on Software Engineering* 32, 12 (2006), 971–987.
- Xuan Bach D. Le, David Lo, and Claire Le Goues. 2016. History driven program repair. In *2016 IEEE 23rd International Conference on Software Analysis, Evolution and Reengineering (SANER)*. 213–224. doi:10.1109/SANER.2016.76
- Claire Le Goues, ThanhVu Nguyen, Stephanie Forrest, and Westley Weimer. 2012. Genprog: A generic method for automatic software repair. *IEEE Trans. Software Eng.* 38, 1 (2012), 54–72.
- Claire Le Goues, Michael Pradel, and Abhik Roychoudhury. 2019. Automated program repair. *Commun. ACM* 62, 12 (2019), 56–65. doi:10.1145/3318162
- Caroline Ext Lemieux, Jeevana Priya Ext Inala, Shuvendu K. Lahiri, and Koushik Sen. 2023. Codamosa: Escaping coverage plateaus in test generation with pre-trained large language models. In *45th International Conference on Software Engineering (ICSE)*.
- Yi Li, Shaohua Wang, and Tien N. Nguyen. 2020. Dlfix: Context-based code transformation learning for automated program repair. In *Proceedings of the International Conference on Software Engineering (ICSE)*.
- Kui Liu, Anil Koyuncu, Dongsun Kim, and Tegawendé F. Bissyandé. 2019. Tbar: revisiting template-based automated program repair. In *Proceedings of the 2019 ACM SIGPLAN International Symposium on Software Testing and Analysis (ISSTA)*. ACM, 31–42. doi:10.1145/3293882.3330577
- Yi Ext Liu, Sergey Mechtaev, Pavle Subotić, and Abhik Roychoudhury. 2023. Program repair guided by datalog-defined static analysis. In *Proceedings of the 2023 ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE)*. 1216–1228.
- Fan Long and Martin Rinard. 2016. Automatic patch generation by learning correct code. In *Proceedings of the 43rd Annual ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages (POPL)*. 298–312.
- Thibaud Lutellier, Hung Viet Ext Pham, Lawrence Pang, Yitong Ext Li, Melissa Wei, and Lin Tan. 2020. Coconut: combining context-aware neural translation models using ensemble for program repair. In *Proceedings of the 2020 ACM SIGPLAN International Symposium on Software Testing and Analysis (ISSTA)*. ACM, 101–114. doi:10.1145/3395363.3397369
- Alexandru Marginean, Johannes Bader, Satish Chandra, Mark Harman, Yue Ext Jia, Ke Ext Mao, Alexander Mols, and Andrew Scott. 2019. Sapfix: Automated end-to-end repair at scale. In *Proceedings of the International Conference on Software Engineering: Software Engineering in Practice (ICSE-SEIP)*.
- Sergey Mechtaev, Jooyong Yi, and Abhik Roychoudhury. 2016. Angelix: Scalable multiline program patch synthesis via symbolic analysis. In *Proceedings of the 38th International Conference on Software Engineering*. 691–701.
- Grégoire Mialon, Roberto Dessi, Maria Lomeli, Christoforos Ext Nalmpantis, Ram Ext Pasunuru, Roberta Ext Raileanu, Baptiste Rozière, Timo Ext Schick, Jane Dwivedi-Yu, Asli Ext Celikyilmaz, Edouard Grave, Yann LeCun, and Thomas Scialom. 2023. Augmented language models: a survey. *CoRR abs/2302.07842* (2023). doi:10.48550/arXiv.2302.07842
- Hoang Duong Thien Nguyen, Dawei Qi, Abhik Roychoudhury, and Satish Chandra. 2013. Semfix: program repair via semantic analysis. In *Proceedings of the 35th International Conference on Software Engineering (ICSE)*. 772–781.
- Shishir G. Patil, Tianjun Ext Zhang, Xin Ext Wang, and Joseph E. Gonzalez. 2023. Gorilla: Large language model connected with massive apis. *CoRR abs/2305.15334* (2023). doi:10.48550/arXiv.2305.15334
- Gabriel Ext Ryan, Sonal Ext Jain, Mingyuan Ext Shang, Siyu Ext Ext Ext Wang, Xiaofei Ext Ma, Murali Krishna Ramanathan, and Baishakhi Ray. 2024. Code-aware prompting: A study of coverage guided test generation in regression setting using llm. In *ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering (FSE)*.
- Georgios Ext Sakkas, Madeline Ext Endres, Philip J. Guo, Westley Weimer, and Ranjit Jhala. 2022. Seq2parse: neurosymbolic parse error repair. *Proc. ACM Program. Lang.* 6, OOPSLA2 (2022), 1180–1206. doi:10.1145/3563330
- Max Ext Schäfer, Sarah Ext Nadi, Ali Ext Eghbali, and Frank Tip. 2024. An empirical evaluation of using large language models for automated unit test generation. *IEEE Trans. Software Eng.* 50, 1 (2024), 85–105. doi:10.1109/TSE.2023.3334955
- Timo Schick, Jane Dwivedi-Yu, Roberto Dessi, Roberta Ext Raileanu, Maria Lomeli, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. 2023. Toolformer: Language models can teach themselves to use tools. *CoRR abs/2302.04761* (2023). doi:10.48550/arXiv.2302.04761
- Disha Ext Ext Shrivastava, Hugo Larochelle, and Daniel Tarlow. 2023. Repository-level prompt generation for large language models of code. In *International Conference on Machine Learning (ICML)*. PMLR, 31693–31715.
- André Ext Silva, Nicolas Saavedra, and Martin Monperrus. 2024. Github-java: A reproducible benchmark of recent java bugs. In *Proceedings of the 21st International Conference on Mining Software Repositories*.
- Daniel Tarlow, Subhdeep Ext Moitra, Andrew Rice, Zimin Chen, Pierre-Antoine Ext Manzagol, Charles Sutton, and Edward Aftandilian. 2020. Learning to fix build errors with graph2diff neural networks. In *ICSE '20: 42nd International Conference on Software Engineering, Workshops*. ACM, 19–20. doi:10.1145/3387940.3392181

- Amit Ext Thakur, George Ext Tsoukalas, Yiyu Ext Wen, Ji Ext Xin, and Swarat Chaudhuri. 2024. An in-context learning agent for formal theorem-proving. *arXiv preprint* (2024). <https://arxiv.org/abs/2310.04353>
- Michele Tufano, Juri Ext Pantiuchina, Cody Watson, Gabriele Bavota, and Denys Poshyvanyk. 2019. On learning meaningful code changes via neural machine translation. In *Proceedings of the International Conference on Software Engineering (ICSE)*. 25–36. <https://dl.acm.org/citation.cfm?id=3339509>
- Ruan van Tonder and Claire Le Goues. 2018. Static automated program repair for heap properties. In *Proceedings of the International Conference on Software Engineering (ICSE)*. 151–162. doi:10.1145/3180155.3180250
- Marko Ext Vasic, Aditya Ext Kanade, Petros Maniatis, David Bieber, and Rishabh Singh. 2019. Neural program repair by jointly learning to localize and repair. In *International Conference on Learning Representations (ICLR)*.
- Ke Ext Wang, Rishabh Ext Singh, and Zhendong Su. 2018. Search, align, and repair: data-driven feedback generation for introductory programming exercises. In *Proceedings of the 39th ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI)*. 481–495.
- Lei Wang, Chang Ext Ma, Xuefeng Ext Feng, Zeyu Ext Zhang, He Ext Yang, Jiacheng Ext Zhang, Zhiyuan Ext Chen, Jiakai Tang, Xu Ext Chen, Yankai Lin, Wayne Xin Zhao, Zhewei Wei, and Ji-Rong Wen. 2023. A survey on large language model based autonomous agents. *arXiv preprint* (2023).
- Jason Wei, Xuezhi Ext Wang, Dale Schuurmans, Maarten Bosma, Fei Ext Xia, Ed Ext Chi, Quoc V. Le, Denny Zhou, et al. 2022. Chain-of-thought prompting elicits reasoning in large language models. In *Advances in Neural Information Processing Systems*, Vol. 35. 24824–24837.
- Chunqiu Steven Xia, Matteo Paltenghi, Jifeng L. Ext Tian, Michael Pradel, and Lingming Zhang. 2024. Fuzz4all: Universal fuzzing with large language models. In *Proceedings of the International Conference on Software Engineering (ICSE)*.
- Chunqiu Steven Xia, Yuxiang Wei, and Lingming Zhang. 2023. Automated program repair in the era of large pre-trained language models. In *2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE)*. 1482–1494.
- Chunqiu Steven Xia and Lingming Zhang. 2023. Keep the conversation going: Fixing 162 out of 337 bugs for \$0.42 each using ChatGPT. *arXiv preprint* (2023).
- Jifeng Xuan, Matias Martinez, Favio Demarco, Maxime Clement, Sebastian Lamelas Marcote, Thomas Durieux, Daniel Le Berre, and Martin Monperrus. 2016. Nopol: Automatic repair of conditional statement bugs in java programs. *IEEE Transactions on Software Engineering* 43, 1 (2016), 34–55.
- Dehai Ext Yang, Xiaoguang Ext Mao, Li Ext Chen, Xuehai Ext Xu, Yan Ext Lei, David Lo, and Jianjun Ext He. 2022. Transplantfix: Graph differencing-based code transplantation for automated program repair. In *Proceedings of the 37th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. 107:1–107:13. doi:10.1145/3551349.3556893
- John Ext Yang, Carlos E. Ext Jimenez, Kilian Ext Lieret, Shunyu Ext Ext Yao, Alexander Ext Wettig, Karthik Ext Narasimhan, and Ofir Ext Press. 2024. SWE-agent: Agent-computer interfaces enable automated software engineering. *arXiv preprint* (2024).
- He Ye, Matias Martinez, Xiapu Ext Luo, Tao Ext Zhang, and Martin Monperrus. 2022b. Selfapr: Self-supervised program repair with test execution diagnostics. In *Proceedings of the 37th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. 92:1–92:13. doi:10.1145/3551349.3556926
- He Ye, Matias Martinez, and Martin Monperrus. 2022a. Neural program repair with execution-based backpropagation. In *Proceedings of the International Conference on Software Engineering (ICSE)*.
- He Ye and Martin Monperrus. 2024. Iter: Iterative neural repair for multi-location patches. In *Proceedings of the International Conference on Software Engineering (ICSE)*.
- Ting Ext Yu and Michael Pradel. 2017. Pinpointing and repairing performance bottlenecks in concurrent programs. *Empirical Software Engineering (EMSE)* (2017), 1–38.
- Andreas Zeller. 2009. *Why programs fail: a guide to systematic debugging*. Elsevier.
- Quanjin Ext Zhang, Chunrong Ext Fang, Yu Ext Ma, Weisong Ext Sun, and Zhenyu Ext Chen. 2023. A survey of learning-based automated program repair. *ACM Transactions on Software Engineering and Methodology* 33, 2 (2023), 1–69.
- Xiaoling Ext Zhang, Juan Ext Zhai, Shiqing Ext Ma, and Chao Ext Shen. 2021. AUTOTRAINER: an automatic DNN training problem detection and repair system. In *43rd IEEE/ACM International Conference on Software Engineering (ICSE)*. IEEE, 359–371. doi:10.1109/ICSE43902.2021.00043
- Yuntong Ext Zhang, Haifeng Ext Ruan, Zhiyu Ext Fan, and Abhik Roychoudhury. 2024. Autocoderover: Autonomous program improvement. *arXiv preprint* (2024).
- Qihao Ext Zhu, Zeyu Ext Sun, Yaomian Xiao, Wenjie Ext Zhang, Kang Ext Yuan, Yingfei Xiong, and Lu Zhang. 2021. A syntax-guided edit decoder for neural program repair. In *Proceedings of the 29th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE)*. ACM, 341–353. doi:10.1145/3468264.3468544